

University of Applied Sciences Hamburg	Digital Systems	 Hochschule für Angewandte Wissenschaften Hamburg <i>Hamburg University of Applied Sciences</i>
Group/Table No: 1/7	Lab 3 Room 8.01	Efe Kardes Matriculation No: 2757072
Date: 11/12/2025		Emir Avni Sahin Matriculation No: 2756541
Professor: Peter Schulz		Sajjad Rahman Matriculation No: 2613734
Signal Arithmetics		

Report History

Report 1		Date		Remarks
	received			
	checked			
	result	o.k.		
		n.g.		

Prof.

Contents

1	Introduction	3
2	Summary of Preparatory Tasks	3
2.1	Preparation Task 1: Stimuli Data Generation	3
2.2	Preparation Task 2: Control Logic Definition	3
2.3	Preparation Task 3: Hardware Constraint Mapping	5
3	System Architecture and Implementation	5
3.1	Clock Generation	5
3.2	Datapath Logic (ALU)	6
3.3	Gain Unit	6
4	Laboratory Setup	6
4.1	Signal Connections	6
4.2	Control Hardware	7
5	Results and Discussion	7
5.1	Task 1: Basic Functionality	7
5.1.1	Implementation	7
5.1.2	Operational Verification	7
5.2	Task 2.1: System Baseline and Timing Behavior	8
5.2.1	Configuration A	8
5.2.2	Signal Integrity and Channel Verification	8
5.2.3	Verification with Different Waveform Shapes	8
5.2.4	Amplitude Variation Tests	9
5.2.5	Latency Measurement	10
6	Lab Task 2.2 – Configuration A: Arithmetic Behavior	10
6.1	Objective	10
6.2	Setup	10
6.3	Verification of Arithmetic Operations	11
6.4	Verification of Gain Modes	11
7	Lab Task 2.2 – Configuration B: Overflow and Saturation	12
7.1	Objective	12
7.2	Setup	12
7.3	Overflow Behavior and Saturation Effect	12
7.4	Negative Saturation Indicator	13
7.5	Zero-Crossing Detection	14
7.6	Discussion	15
7.7	Conclusion	15
8	Summary	15
9	Literature	16
10	Declaration of AI Usage	16

11 Appendix	16
11.1 VHDL Source Code	16
11.2 Pin Constraints (XDC)	19
11.2.1 IOM Connector 1 (Left Channel Controls)	19
11.2.2 IOM Connector 4 (Right Channel Controls)	19
11.3 Python Generation Script	20

1 Introduction

The goal of this laboratory experiment was to design and implement a digital audio processing system on the Modsys Artix-7 FPGA board. The main task was to create a "datapath" that receives serial audio data, manipulates it such as mixing channels or changing the volume and sends it back out.

In this project, we worked with a VHDL entity called `codec_interface`. Since the audio CODEC on the board communicates one bit at a time, but the FPGA needs full numbers to do math, we had to build several logic blocks to handle the data flow:

- **Clock Dividers:** We created different clock speeds from the main 100 MHz system clock to synchronize the audio sampling (48.8 kHz) and the data transfer.
- **Serial-to-Parallel Conversion:** We used SIPO and PISO blocks to convert the incoming single-wire data into 16-bit signed integers for processing, and then back to serial for the output.
- **Arithmetic Unit (ALU):** This is where we manipulated the data. We implemented logic to perform operations like adding the Left and Right channels, subtracting them, or inverting the signal. We also added "Saturation" logic. This ensures that if a calculation result gets too large (overflow), it clamps to the maximum value instead of wrapping around, which would corrupt the signal.
- **Gain Control:** To simulate volume control, we used a Gain Unit. Instead of using a complex multiplier, we used efficient bit-shifting. Shifting bits to the left makes the value larger ($2\times$), and shifting to the right makes it smaller ($0.5\times$ or $0.25\times$).

Verification was done purely by measuring the electronic signals. We used a function generator to provide specific inputs (Sine and Square waves) and an oscilloscope to visually inspect the output. This allowed us to measure the system delay and verify that our ALU and Gain logic were modifying the waveforms correctly.

2 Summary of Preparatory Tasks

The preparation for this laboratory was divided into three distinct tasks to ensure the VHDL design could be verified in simulation and successfully deployed on the hardware.

2.1 Preparation Task 1: Stimuli Data Generation

To verify the VHDL logic before synthesis, we created a set of synthetic test vectors. A Python script was written to generate a text file (`stimuli.txt`) containing 100 samples of mixed signals.

2.2 Preparation Task 2: Control Logic Definition

We defined the binary control vectors required to operate the Arithmetic Unit (AU) and Gain Unit (GU) as follows:

- **Operation Vector (3-bit):** We defined 8 distinct modes for the ALU. These included basic mixing operations like Addition ($L + R$) and Subtraction ($L - R$), as well as channel routing modes to pass the Left or Right signals through unchanged.

Table 1: Operation Control Vector Encoding (3-bit)

Code (Binary)	Symbol	Function
000	$Y' = L + R$	Addition (Mix)
001	$Y' = L - R$	Subtraction
010	$Y' = R - L$	Reverse Subtraction
011	$Y' = L$	Pass Left Channel
100	$Y' = R$	Pass Right Channel
101	$Y' = -L$	Invert Left Channel
110	$Y' = -R$	Invert Right Channel
111	N/A	Reserved / Output Zero

- **Saturation Mode (1-bit):** We defined a single bit to control overflow protection. Setting this bit to 0 allows standard wrap-around behavior, while setting it to 1 enables the clipping logic, which clamps values to the maximum or minimum integer limit.

Table 2: Saturation Mode Control (1-bit)

Code	Mode	Description
0	Unsaturated	Standard overflow
1	Saturated	Clipping behavior

- **Gain Vector (2-bit):** Mapped to bit-shift operations to control volume. As detailed in Table 3, we utilized logical left shifts for amplification ($2\times$) and arithmetic right shifts for attenuation ($0.5\times$, $0.25\times$) to ensure the sign bit is preserved.

Table 3: Gain Mode Control Encoding (2-bit)

Code	Factor	Operation	Implementation
00	1.0x	Unity Gain	No Shift
01	2.0x	Amplification	Logical Shift Left (<code>sll 1</code>)
10	0.5x	Attenuation	Arithmetic Shift Right (<code>sra 1</code>)
11	0.25x	Strong Attenuation	Arithmetic Shift Right (<code>sra 2</code>)

2.3 Preparation Task 3: Hardware Constraint Mapping

The final preparation task involved mapping the VHDL ports to the physical switches and LEDs on the IOM boards. The pin assignment plan used for the laboratory is detailed in Table 4.

Table 4: IOM Board Switch and LED Assignments

Component	Left Channel (Conn 1)	Right Channel (Conn 4)
Switches (Inputs)		
Switch 0	Global Enable (<code>enable</code>)	Unused / NC
Switch 1	Op Mode Bit 0 (<code>op_mode_L[0]</code>)	Op Mode Bit 0 (<code>op_mode_R[0]</code>)
Switch 2	Op Mode Bit 1 (<code>op_mode_L[1]</code>)	Op Mode Bit 1 (<code>op_mode_R[1]</code>)
Switch 3	Op Mode Bit 2 (<code>op_mode_L[2]</code>)	Op Mode Bit 2 (<code>op_mode_R[2]</code>)
Switch 4	Saturation En (<code>sat_mode_L</code>)	Saturation En (<code>sat_mode_R</code>)
Switch 5	Gain Bit 0 (<code>gain_mode_L[0]</code>)	Gain Bit 0 (<code>gain_mode_R[0]</code>)
Switch 6	Gain Bit 1 (<code>gain_mode_L[1]</code>)	Gain Bit 1 (<code>gain_mode_R[1]</code>)
LEDs (Outputs)		
LED 0	Zero Cross Rise (<code>L_H_L</code>)	Zero Cross Rise (<code>L_H_R</code>)
LED 1	Zero Cross Fall (<code>H_L_L</code>)	Zero Cross Fall (<code>H_L_R</code>)
LED 2	Pos Saturation (<code>P_Sat_L</code>)	Pos Saturation (<code>P_Sat_R</code>)
LED 3	Neg Saturation (<code>N_Sat_L</code>)	Neg Saturation (<code>N_Sat_R</code>)

Note: In addition to the signal mappings above, we configured **LED 9** on the IOM extension board to function as a 1 Hz "Heartbeat" monitor. This blinking LED provided visual confirmation that the system clock and bit counters were operating correctly.

3 System Architecture and Implementation

The entire design was implemented in a single VHDL entity called `codec_interface`. This module handles everything from generating clocks to processing the audio data.

3.1 Clock Generation

The FPGA runs on a 100 MHz master clock. To communicate with the audio chip and process data, we created four slower clocks using counters:

- **LRC (48 kHz):** This is the audio sample rate. It is created by dividing the master clock by 2048.
- **BIT (1.56 MHz):** This clock shifts the data in and out. It is the master clock divided by 64.
- **SYS (12.5 MHz):** This is the internal processing clock. It runs faster than the audio rate to ensure calculations are finished before the next sample arrives.
- **Heartbeat (1 Hz):** We added a very slow clock to blink LED 9, just to prove the system is running.

3.2 Datapath Logic (ALU)

The Arithmetic Logic Unit (ALU) was written as a VHDL procedure so we could use the exact same logic for both Left and Right channels.

- **Signed Arithmetic:** The system treats the 16-bit audio samples as signed integers.
- **Saturation Logic:** To handle overflow, the ALU actually calculates with **17 bits**. If the result is too big for 16 bits, the logic checks the "saturation enable" switch. If enabled, it forces the output to the maximum positive or minimum negative value instead of letting it wrap around.

3.3 Gain Unit

Instead of using a complex multiplier component, we implemented volume control using bit-shifting functions:

- **Amplification:** We used a "Logic Shift Left" (`sll`) to multiply the signal by 2.
- **Attenuation:** We used an "Arithmetic Shift Right" (`sra`) to divide the signal by 2 or 4. Using the arithmetic shift is important because it preserves the sign bit (positive/negative) of the audio.

4 Laboratory Setup

The experimental setup was built around the Modsys Artix-7 FPGA board, equipped with an Audio CODEC module and two IOM extension boards for user control.

4.1 Signal Connections

We used two function generators to provide distinct input signals for the Left and Right channels. These inputs were connected to the CODEC's "Line In" port, while the CODEC's "Line Out" port was connected back to the oscilloscope for analysis.

The specific channel mapping on the oscilloscope was configured as follows:

- **Channel 1 (Yellow) - Input** Connected to the output of Function Generator 1.
 - Signal: Sine Wave (100 Hz, 1 V_{pp}).
 - Route: Goes to CODEC Left Line-In.
- **Channel 2 (Green) - Input** Connected to the output of Function Generator 2.
 - Signal: Square Wave (100 Hz, 1 V_{pp}).
 - Route: Goes to CODEC Right Line-In.
- **Channel 3 (Orange) - Output:** Probed the CODEC Left Line-Out.
- **Channel 4 (Blue) - Output:** Probed the CODEC Right Line-Out.

4.2 Control Hardware

To control the internal logic (ALU and Gain), we used the switches on the IOM extension boards.

- **IOM Connector 1:** Controlled the Left Channel parameters (Op Mode, Gain, Saturation).
- **IOM Connector 4:** Controlled the Right Channel parameters.

The exact switch mapping corresponds to the definitions listed in Table 4 in the Preparation section.

5 Results and Discussion

5.1 Task 1: Basic Functionality

The objective of Task 1 was to implement the circuit on the FPGA and verify its basic operation using an oscilloscope.

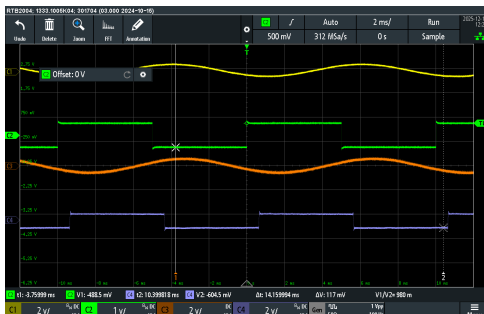
5.1.1 Implementation

We created a new RTL project in Vivado, imported the VHDL source files and the corresponding XDC constraints for the Audio CODEC and IOM boards. After synthesis and implementation, the bitstream was successfully generated and loaded onto the Artix-7 FPGA. The complete source code used for this implementation can be found in the Appendix.

5.1.2 Operational Verification

To verify the basic functionality, we connected two independent function generators to the CODEC inputs:

- Left Channel: Sine Wave (100 Hz).
- Right Channel: Square Wave (100 Hz).



(a) **Enable ON (Switch 0 = 1):** System active. Outputs (Ch3/Ch4) replicate inputs.



(b) **Enable OFF (Switch 0 = 0):** System reset. Outputs forced to zero (muted).

Figure 1: Verification of Basic Operation

5.2 Task 2.1: System Baseline and Timing Behavior

The objective of Lab Task 2.1 was to analyze the timing behavior and verify the correct operation of the complete audio signal chain under a baseline configuration in which pass-through configuration is applied. This configuration serves as a reference point for all subsequent experiments by validating signal integrity, correct channel routing, and overall system latency.

5.2.1 Configuration A

For this task, the system was configured as follows:

- Left Channel Arithmetic Unit: $Y' = L$
- Right Channel Arithmetic Unit: $Y' = R$
- Saturation Mode: Disabled
- Gain Mode: Unity gain ($Y = Y'$) on both channels

With this configuration, the Channel Arithmetic Units are active but do not modify the signals. The system therefore operates in a logical pass-through mode. Any observed delay or distortion is caused by internal buffering, serial-to-parallel and parallel-to-serial conversion, and the ADC/DAC processing stages of the audio CODEC.

5.2.2 Signal Integrity and Channel Verification

To verify correct operation, different waveform shapes, frequencies, and amplitudes were applied independently to the left and right audio input channels using two function generators. The resulting output signals were observed on the oscilloscope.

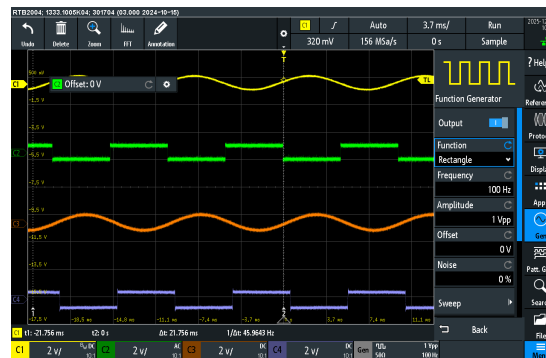


Figure 2: Baseline pass-through verification. The output signals replicate the input waveforms for both channels.

Figure 2 confirms correct channel routing, as the left and right outputs accurately reproduce the corresponding input waveforms without unintended mixing or distortion.

5.2.3 Verification with Different Waveform Shapes

Additional measurements were performed using non-sinusoidal waveforms to further validate robustness of the datapath.



Figure 3: Triangle waveform applied to the left channel. The arithmetic unit is configured for pass-through operation.

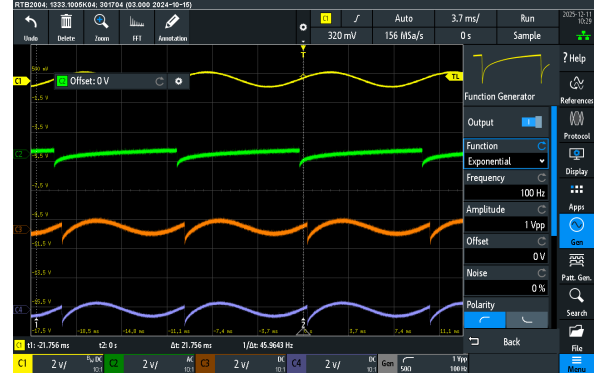
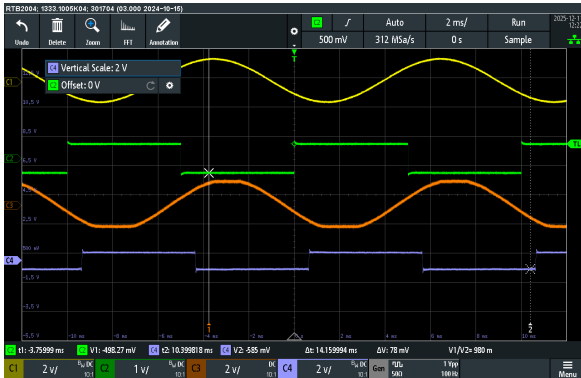


Figure 4: Exponential waveform applied to the right channel. The arithmetic unit is configured for pass-through operation and the output reproduces the input waveform.

The measurements demonstrate that the datapath maintains signal integrity for different waveform types, including triangular and exponential signals.

5.2.4 Amplitude Variation Tests

To further confirm stable operation, the input signal amplitude was increased while maintaining the same pass-through configuration.



(a) Pass-through behavior with increased input amplitude. No distortion observed.



(b) Triangle waveform with increased amplitude, confirming linear behavior.

Figure 5: Amplitude Variation Tests (Task 2.1)

These results confirm that the datapath operates linearly and remains stable across different amplitude levels in baseline mode.

5.2.5 Latency Measurement

The total system latency was measured by comparing a rising edge of the input signal with the corresponding rising edge at the output signal.

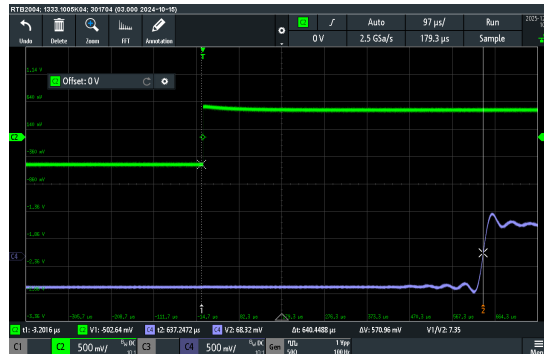


Figure 6: Total signal delay measurement.

Given the sampling frequency of 48.8 kHz, the sampling period is:

$$T_s = \frac{1}{48.8 \text{ kHz}} \approx 20.48 \mu s$$

Accuracy Discussion: In the previous laboratory session (direct CODEC loopback), the baseline latency was measured at approximately 635 μs . The current measurement of 640.45 μs shows an increase of $\approx 5.45 \mu s$.

The additional FPGA logic may introduce a delay on the order of one sampling period, achieving this level of measurement accuracy on the oscilloscope is challenging due to cursor placement resolution and signal noise. The small discrepancy between the measured increase (5.45 μs) and the theoretical increase (20.48 μs) is attributed to these measurement tolerances.

6 Lab Task 2.2 – Configuration A: Arithmetic Behavior

6.1 Objective

The goal of Configuration A was to verify the Arithmetic Unit's ability to perform real-time mathematical operations ($L + R$, $L - R$) and to validate the Gain Unit's bit-shifting logic.

6.2 Setup

Two signal generators were connected to the system inputs:

- **Left Channel (Input):** Sine Wave (100 Hz, 1 V_{pp}).
- **Right Channel (Input):** Square Wave (100 Hz, 1 V_{pp}).

The FPGA was initially configured with Saturation disabled and Unity Gain.

6.3 Verification of Arithmetic Operations

We configured the Left Channel for Addition ($Y' = L + R$) and the Right Channel for Subtraction ($Y' = L - R$).



Figure 7: Arithmetic verification. CH3 (orange): $L + R$. CH4 (blue): $L - R$.

As shown in Figure 7, the output signals confirm the logic:

- **Ch3 (Addition):** The Sine wave is shifted up/down in phase with the Square wave.
- **Ch4 (Subtraction):** The Sine wave shifts in the opposite direction, confirming the subtraction logic.

6.4 Verification of Gain Modes

Following the arithmetic test, we varied the Gain Control switches to verify the bit-shifting logic. We captured the output waveforms for all four gain settings: Unity ($1\times$), Amplification ($2\times$), and Attenuation ($0.5\times$, $0.25\times$).



Gain 1x (00): Unity gain. Signal unchanged.



Gain 2x (01): Logic shift left. Amplitude doubles.

Figure 8: Gain Unit Verification (Part 1): Amplification Modes



Gain 0.5x (10): Arithmetic shift right by 1.



Gain 0.25x (11): Arithmetic shift right by 2.

Figure 9: Gain Unit Verification (Part 2): Attenuation Modes

Observation: The waveforms in Figure 8 and Figure 9 confirm that the logical left shift correctly doubles the signal amplitude, while the arithmetic right shifts attenuate the signal while preserving the signed integer format (polarity).

7 Lab Task 2.2 – Configuration B: Overflow and Saturation

7.1 Objective

The objective of Configuration B is to experimentally investigate arithmetic overflow behavior and the effect of saturation arithmetic in the implemented Channel Arithmetic Units. In addition, the correct operation of the saturation indicator signals and zero-crossing detection is verified using oscilloscope measurements.

7.2 Setup

The experimental setup used for Configuration B is identical to the setup applied in Configuration 2.2.A. The complete FPGA-based signal processing system with audio CODEC interface, Channel Arithmetic Units, and IOM-based control and status indication was used without any structural changes.

To provoke arithmetic overflow, the amplitudes of the input signals generated by the function generators were gradually increased during operation. The resulting behavior of the processed output signals and the corresponding status indicators was observed using an oscilloscope. This procedure allowed a direct comparison between unsaturated and saturated arithmetic behavior under identical signal conditions.

7.3 Overflow Behavior and Saturation Effect

With the saturation switch set to logic level 0, saturation was disabled. Under this condition, arithmetic overflow was observed on channel CH4. Channel CH2 was used to monitor the positive saturation indicator.

Figure 10 shows the behavior with saturation disabled. Due to the absence of clipping, the arithmetic result exhibits wrap-around behavior at the numerical limits, which is characteristic of unsaturated two's complement arithmetic. The positive saturation indicator remains inactive in this mode.



Figure 10: Positive saturation indicator on CH2 with saturation switch set to 0 (disabled)

After enabling saturation by setting the saturation switch to logic level 1, the same signal conditions were applied. Figure 11 shows the resulting behavior.

In contrast to the unsaturated case, the arithmetic result is now limited at the upper numerical bound. The positive saturation indicator on CH2 becomes active when the processed signal reaches this limit, confirming correct saturation behavior.



Figure 11: Positive saturation indicator on channel 2 with saturation switch set to 1 (enabled)

7.4 Negative Saturation Indicator

To verify negative saturation behavior, channel 2 was used to monitor the negative saturation indicator.

Figure 12 shows that the negative saturation LED is asserted when the processed signal reaches the lower numerical limit. The indicator is active for a single sample period, in accordance with the specified behavior of the arithmetic unit.



Figure 12: Negative saturation indicator observed on CH2

7.5 Zero-Crossing Detection

Zero-crossing detection was verified using a sinusoidal input signal with an amplitude of 1 V_{pp} and a frequency of 100 Hz . Due to the relatively low signal frequency and the high internal system clock rate, the zero-crossing event occurs within a very short time interval and cannot be resolved by directly zooming into the complete waveform.

Therefore, the oscilloscope zoom functionality was used. The upper part of the oscilloscope display shows the processed sinusoidal signal over a larger time scale, while the lower, zoomed section focuses on the region around the zero-crossing event.

In the measurement, CH3 was connected to the processed output signal, and CH2 was connected to the zero-crossing indicator output. The zoomed view shows that the signal on CH2 is asserted when the processed signal on CH3 changes its sign. This behavior corresponds to a zero crossing of the sinusoidal waveform and occurs synchronously with the system update clock.



Figure 13: Zero-crossing detection using oscilloscope zoom functionality (CH3: processed signal, CH2: zero-crossing indicator)

7.6 Discussion

The measurements obtained in Configuration B clearly demonstrate the difference between unsaturated and saturated arithmetic. With saturation disabled, arithmetic overflow results in wrap-around behavior, which leads to a distortion of the output signal. When saturation is enabled, this behavior is prevented by limiting the output to the representable numerical range.

The positive and negative saturation indicators reliably signal clipping events and are active for a single sample period, allowing precise identification of overflow occurrences. The zero-crossing detection mechanism operates correctly and is synchronized with the sign change of the processed signal, as confirmed by the zoomed oscilloscope measurement.

7.7 Conclusion

Configuration B successfully verified arithmetic overflow behavior, saturation functionality, and status signal generation in the Channel Arithmetic Units. All observed measurements are consistent with the expected behavior and confirm the correct implementation of the arithmetic datapath and associated monitoring logic.

8 Summary

In this laboratory experiment, a complete FPGA-based audio signal processing system was implemented and experimentally verified. The system includes serial audio data handling via an audio CODEC, configurable channel arithmetic units, gain control, saturation logic, and status signal generation.

In Task 1, the basic functionality of the system was validated. The correct operation of the audio datapath, channel routing, and global enable control was confirmed using oscilloscope measurements. The system reliably forwarded input signals to the corresponding outputs and responded correctly to enable and reset conditions.

Task 2.1 established a baseline configuration in which the arithmetic units were configured for pass-through operation. Signal integrity was verified for different waveform types and amplitudes, and a stable and deterministic system latency was measured. This baseline served as a reference for subsequent arithmetic experiments.

In Task 2.2 Configuration A, the correct operation of the arithmetic and gain units was verified. The system correctly performed addition and subtraction of input signals, and the gain unit implemented the specified scaling factors using bit-shift operations.

In Task 2.2 Configuration B, overflow behavior, saturation arithmetic, and zero-crossing detection were examined. Unsaturated arithmetic resulted in wrap-around behavior during overflow, while enabling saturation correctly limited the output signal. Positive and negative saturation indicators reliably signaled clipping events, and zero-crossing detection operated correctly in synchronization with the sign change of the processed signal.

Overall, the experimental results confirm that the implemented audio processing system operates as specified and that all functional blocks of the design perform correctly under the tested conditions.

9 Literature

References

- [1] P. Schulz, *Lab Session on Signal Arithmetics (Rev 1.1)*, Hamburg University of Applied Sciences (HAW Hamburg), 2025.
- [2] Xilinx Inc., *7 Series FPGAs Data Sheet: Overview (DS180)*, 2018. [Online]. Available: <https://www.xilinx.com>
- [3] Texas Instruments / Burr-Brown, *PCM3006 16-Bit, Single-Ended Analog Input/Output Stereo Audio Codec (SBAS089A)*, 2004.
- [4] L. Leutelt, *Lecture Slides and Lab Descriptions on Digital Systems*, Hamburg University of Applied Sciences (HAW Hamburg), 2024.
- [5] IEEE Computer Society, *IEEE Standard VHDL Language Reference Manual (IEEE Std 1076-2008)*, 2009.
- [6] Rohde & Schwarz, *RTB2004 Digital Oscilloscope – User Manual*, Rohde & Schwarz GmbH & Co. KG, 2021.
- [7] Xilinx Inc., *Vivado Design Suite – Synthesis User Guide (UG901)*, 2023.

10 Declaration of AI Usage

During the preparation of this report, generative AI tools were utilized to assist with specific editorial and formatting tasks. Specifically, AI was used for:

- Verifying the document structure and logical flow.
- Checking for grammatical errors, typos, and figure labeling consistency.
- Formatting the LaTeX code and organizing the Appendix and Literature sections.

11 Appendix

11.1 VHDL Source Code

The following listing presents the core logic of the `codec_interface` entity.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity codec_interface is
    generic (
        clk_in_frequency : integer := 100000000;    -- 100 MHz
        LRC_frequency    : integer := 48828;        -- ~48.8 kHz
        BIT_frequency     : integer := 1562500;      -- ~1.56 MHz
        SYS_frequency     : integer := 12500000;     -- 12.5 MHz
        data_width        : natural := 16
    );
```

```

port (
    reset_n, enable, clk_in : in  std_logic;
    clk_LRC, clk_BIT, clk_SYS, clk_d : out std_logic;
    DOUT : out std_logic; DIN : in  std_logic;
    -- Control Inputs
    op_mode_L, op_mode_R : in std_logic_vector(2 downto 0);
    sat_mode_L, sat_mode_R : in std_logic;
    gain_mode_L, gain_mode_R : in std_logic_vector(1 downto 0);
    -- Status Outputs
    P_Sat_L, N_Sat_L, P_Sat_R, N_Sat_R : out std_logic;
    L_H_L, H_L_L, L_H_R, H_L_R : out std_logic
);
end codec_interface;

architecture test of codec_interface is
    -- [Signal Declarations Omitted for Brevity] --

    -----
    -- ALU Core Procedure: Handles Arithmetic & Saturation
    -----

    procedure alu_core(
        a, b : in signed(15 downto 0);
        op : in std_logic_vector(2 downto 0);
        sat_en : in std_logic;
        result : out signed(15 downto 0);
        p_sat, n_sat : out std_logic
    ) is
        variable sum : signed(data_width downto 0);
    begin
        p_sat := '0'; n_sat := '0'; result := (others => '0');

        case op is
            when "000" => -- ADD (L+R)
                sum := (a(15) & a) + (b(15) & b);
                if sat_en = '1' and sum(16) /= sum(15) then -- Overflow
                    if sum(16) = '1' then result := x"8000"; n_sat := '1';
                    else result := x"7FFF"; p_sat := '1'; end if;
                else result := sum(15 downto 0); end if;

            when "001" => -- SUB (L-R)
                sum := (a(15) & a) - (b(15) & b);
                -- [Saturation logic identical to ADD] --

            when "010" => -- SUB Reverse (R-L)
                sum := (b(15) & b) - (a(15) & a);

            when "011" => result := a; -- Pass L
            when "100" => result := b; -- Pass R
            when "101" => result := -a; -- Invert L
            when "110" => result := -b; -- Invert R
            when others => result := (others => '0');
        end case;
    end procedure;

    -----
    -- Gain Function: Implements Bit-Shifting
    -----

```

```

function apply_gain(x : signed; mode : std_logic_vector) return
signed is
begin
    case mode is
        when "00" => return x;           -- 1x
        when "01" => return x sll 1;     -- 2x (Logic Shift Left)
        when "10" => return x sra 1;     -- 0.5x (Arith Shift Right)
        when "11" => return x sra 2;     -- 0.25x (Arith Shift Right)
        when others => return x;
    end case;
end function;

begin
    -----
    -- Main Datapath Process
    -----

    ALUs : process(all)
        variable tmp_res : signed(15 downto 0);
        variable tmp_p, tmp_n : std_logic;
    begin
        -- Left Channel Processing
        alu_core(left_signed_L, right_signed_L, op_mode_L, sat_mode_L,
            tmp_res, tmp_p, tmp_n);
        result_L <= tmp_res; P_Sat_L <= tmp_p; N_Sat_L <= tmp_n;

        -- Right Channel Processing
        alu_core(left_signed_R, right_signed_R, op_mode_R, sat_mode_R,
            tmp_res, tmp_p, tmp_n);
        result_R <= tmp_res; P_Sat_R <= tmp_p; N_Sat_R <= tmp_n;
    end process;

    -- Apply Gain to ALU results
    BRL_data_out_L <= apply_gain(result_L, gain_mode_L);
    BRL_data_out_R <= apply_gain(result_R, gain_mode_R);

    -----
    -- Zero-Crossing Detection Process
    -----

    zero_crossing : process(clk_in, reset_n)
    begin
        if reset_n = '0' then -- [Reset Logic Omitted] --
        elsif rising_edge(clk_in) then
            if enable = '1' and clk_en_SYS_rising = '1' then
                -- Detect sign change: Positive to Negative
                if previous_result_L(15) = '0' and result_L(15) = '1'
                then
                    H_L_L <= '1'; else H_L_L <= '0';
                end if;
                -- [Other edges and Right Channel Logic Omitted] --
                previous_result_L <= result_L;
            end if;
        end if;
    end process;

    -- [Clock Divider and Serial Interface Processes Omitted] --

end architecture;

```

11.2 Pin Constraints (XDC)

11.2.1 IOM Connector 1 (Left Channel Controls)

```
# -----
# INPUTS OF IOM BOARD - CONNECTOR 1
# -----
# Switch 0: Global Enable
set_property PACKAGE_PIN D3 [get_ports {enable}]
set_property IOSTANDARD LVCMOS33 [get_ports {enable}]

# Switches 1-3: Operation Mode Left
set_property PACKAGE_PIN H1 [get_ports {op_mode_L[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {op_mode_L[0]}]
set_property PACKAGE_PIN G1 [get_ports {op_mode_L[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {op_mode_L[1]}]
set_property PACKAGE_PIN F1 [get_ports {op_mode_L[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {op_mode_L[2]}]

# Switch 4: Saturation Enable Left
set_property PACKAGE_PIN E1 [get_ports {sat_mode_L}]
set_property IOSTANDARD LVCMOS33 [get_ports {sat_mode_L}]

# Switches 5-6: Gain Mode Left
set_property PACKAGE_PIN C1 [get_ports {gain_mode_L[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gain_mode_L[0]}]
set_property PACKAGE_PIN C2 [get_ports {gain_mode_L[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gain_mode_L[1]}]

# -----
# OUTPUTS OF IOM BOARD - CONNECTOR 1
# -----
# LEDs 0-1: Zero Crossing Indicators
set_property PACKAGE_PIN D5 [get_ports {L_H_left}]
set_property IOSTANDARD LVCMOS33 [get_ports {L_H_left}]
set_property PACKAGE_PIN F5 [get_ports {H_L_left}]
set_property IOSTANDARD LVCMOS33 [get_ports {H_L_left}]

# LEDs 2-3: Saturation Indicators
set_property PACKAGE_PIN D8 [get_ports {P_Sat_L}]
set_property IOSTANDARD LVCMOS33 [get_ports {P_Sat_L}]
set_property PACKAGE_PIN C7 [get_ports {N_Sat_L}]
set_property IOSTANDARD LVCMOS33 [get_ports {N_Sat_L}]
```

Listing 2: Modsys_Artix7_IOM_Conn_1.xdc

11.2.2 IOM Connector 4 (Right Channel Controls)

```
# -----
# INPUTS OF IOM BOARD - CONNECTOR 4
# -----
# Switches 1-3: Operation Mode Right
```

```

set_property PACKAGE_PIN R5 [get_ports {op_mode_R[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {op_mode_R[0]}]
set_property PACKAGE_PIN R6 [get_ports {op_mode_R[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {op_mode_R[1]}]
set_property PACKAGE_PIN N4 [get_ports {op_mode_R[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {op_mode_R[2]}]

# Switch 4: Saturation Enable Right
set_property PACKAGE_PIN M4 [get_ports {sat_mode_R}]
set_property IOSTANDARD LVCMOS33 [get_ports {sat_mode_R}]

# Switches 5-6: Gain Mode Right
set_property PACKAGE_PIN T3 [get_ports {gain_mode_R[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gain_mode_R[0]}]
set_property PACKAGE_PIN R3 [get_ports {gain_mode_R[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {gain_mode_R[1]}]

# -----
# OUTPUTS OF IOM BOARD - CONNECTOR 4
# -----
# LEDs 0-1: Zero Crossing Indicators
set_property PACKAGE_PIN U9 [get_ports {L_H_R}]
set_property IOSTANDARD LVCMOS33 [get_ports {L_H_R}]
set_property PACKAGE_PIN V5 [get_ports {H_L_R}]
set_property IOSTANDARD LVCMOS33 [get_ports {H_L_R}]

# LEDs 2-3: Saturation Indicators
set_property PACKAGE_PIN V4 [get_ports {P_Sat_R}]
set_property IOSTANDARD LVCMOS33 [get_ports {P_Sat_R}]
set_property PACKAGE_PIN L4 [get_ports {N_Sat_R}]
set_property IOSTANDARD LVCMOS33 [get_ports {N_Sat_R}]
# LED 9: Heartbeat Monitor (1 Hz)
set_property PACKAGE_PIN E18 [get_ports {clk_d}]
set_property IOSTANDARD LVCMOS33 [get_ports {clk_d}]

```

Listing 3: Modsys_Artix7_IOM_Conn_4.xdc

11.3 Python Generation Script

```

import numpy as np

# Generate 20 samples for appendix display
N = 20
t = np.linspace(0, 1, N)

# Signal Generation
sine = 15000 * np.sin(2 * np.pi * 5 * t)
noise = 2000 * np.random.randn(N)
ramp = np.linspace(-20000, 20000, N)

# Combine signals
left = sine + noise
right = ramp

# Clip to 16-bit signed range
left = np.clip(left, -32768, 32767).astype(int)
right = np.clip(right, -32768, 32767).astype(int)

```

```
# Output format
for L, R in zip(left, right):
    print(f"{L}_{R}")
```

Listing 4: Python script used for Stimuli Generation